

Introduction to Social Data Analytics:

Lecture 3 Part 2

Load in the Analysis Sample

- ▶ Let's begin by loading in our analysis sample that has information on traffic stops and sunset times

```
use sd_analysis_sample.dta, clear
```

- ▶ Reminder: we want to focus on times in which it is sometimes light out and sometimes dark
- ▶ To start, we will restrict to stops that occur between 6:30 PM and 7:00 PM (this will be harder than you might think!)

Date-times in Stata

- ▶ In Stata, times of the day are often held in terms of milliseconds from midnight
 - ▶ You could store it as a string variables, but you can't add string variables
 - ▶ Often useful to hold times in formats that can be added or subtracted (i.e. if I add an hour to 6:00 PM the result should be 7:00 PM)

6:30 in terms of milliseconds from midnight

- ▶ First, let's summarize our stop_time variable to see how it is stored

```
sum stop_time
```

Variable	Obs	Mean	Std. dev.	Min	Max
-----+-----					
stop_time	381,149	4.70e+07	2.26e+07	0	8.63e+07

- ▶ It is stored in terms of milliseconds from midnight (so 0 is midnight exactly and 1 hour later (1:00 AM) would be $60 \cdot 60 \cdot 1,000 = 3,600,000$)
- ▶ So how many milliseconds until 6:30 PM?

$$\underbrace{18}_{\text{hours}} \cdot \underbrace{3,600,000}_{\text{ms per hour}} + \underbrace{1,800,000}_{\text{ms per half hour}} = 66,600,000$$

- ▶ So we should restrict to stops that happen in the range [66,600,000, 68,400,000]

An Easier Way

- ▶ An easier way is to use the `clock()` command, which you can use to translate times into milliseconds from midnight automatically
- ▶ For example, to see how many milliseconds from midnight 6:30 PM is type

```
di clock("18:30:00", "hms")
```

66600000

- ▶ Where "18:30:00" is 6:30 PM on a 24-hour clock
- ▶ "hms" tells Stata that time should be interpreted as hours, minutes, seconds

Restricting to 6:30-7:00

- To restrict to 6:30-7:00 PM we can type

```
/* drop if before 6:30 PM */  
drop if stop_time<clock("18:30:00","hms")  
/* drop if after 7:00 PM */  
drop if stop_time>clock("19:00:00","hms")
```

(296,355 observations deleted)

(76,052 observations deleted)

- Now let's see how many stops we have left and what times they were made

```
sum stop_time
```

Variable	Obs	Mean	Std. dev.	Min	Max
stop_time	8,742	6.75e+07	632710.1	6.66e+07	6.84e+07

Constructing dark vs. light indicators

- ▶ We will define a stop as occurring in the dark if it occurs after dusk

```
gen dark = (stop_time>dusk)
```

- ▶ We will define a stop as occurring in the light if it is before sunset

```
gen light = (stop_time<sunset)
```

Dropping Twilight

- ▶ Some stops occur after sunset, but before dusk, making it unclear if the officer could observe the driver's race
- ▶ We need to drop these cases before proceeding with the veil of darkness test
- ▶ In our case, since we have already defined `light` and `dark`, we can use these variables to drop stops that happen in between `light` and `dark`

```
drop if light==0 & dark==0
```

(414 observations deleted)

Comparison

- ▶ We need to compare the composition of traffic stops in times of sunlight vs. darkness
- ▶ Reminder: we can use the `tab` command to retrieve the composition of traffic stops by race

```
tab subject_race
```

subject_race	Freq.	Percent	Cum.
Asian/Pacific Islander	801	9.16	9.16
Black	1,144	13.09	22.25
Hispanic	2,981	34.10	56.35
Other	589	6.74	63.09
White	3,227	36.91	100.00
Total	8,742	100.00	

Stop Composition During Sunlight

- If we want the composition, but restrict to stops in the sunlight, we can add a conditional statement to our code

```
tab subject_race if dark==0
```

subject_race	Freq.	Percent	Cum.
Asian/Pacific Islander	429	8.80	8.80
Black	649	13.31	22.10
Hispanic	1,662	34.08	56.18
Other	347	7.12	63.30
White	1,790	36.70	100.00
Total	4,877	100.00	

Stop Composition During Dark

- Now let's look at the composition of stops when it is dark

```
tab subject_race if dark==1
```

subject_race	Freq.	Percent	Cum.
-----+-----			
Asian/Pacific Islander	372	9.62	9.62
Black	495	12.81	22.43
Hispanic	1,319	34.13	56.56
Other	242	6.26	62.82
White	1,437	37.18	100.00
-----+-----			
Total	3,865	100.00	

Comparisons

- ▶ Reminder: we are checking to see if certain races are much less likely to be stopped in darkness relative to sunlight
 - ▶ If this is true, then this is good evidence that this race is being targeted during sunlight hours
- ▶ From the statistics we've shown so far, the composition of traffic stops seems relatively similar during sunlight vs. after dark
- ▶ But it is a little difficult to absorb all the information from these multiple tables
- ▶ To really get the information across, it might be useful to put this information into a figure

Hypothetical Example

- Imagine we are working with the dataset below

	student	school	gpa
1	1	A	3.3
2	2	A	3.2
3	3	B	2.9
4	4	B	3

- Our first goal is to create a bar graph that shows the average GPA for each school

Graph Bar

- ▶ The command `graph bar` is used to make bar graphs in Stata
- ▶ There are lots of ways this command can be altered, so make sure to look at the help file!
- ▶ To start, we are going to create a bar graph that shows average GPA for each school

Basic Syntax

- ▶ The basic syntax for the `graph bar` command is

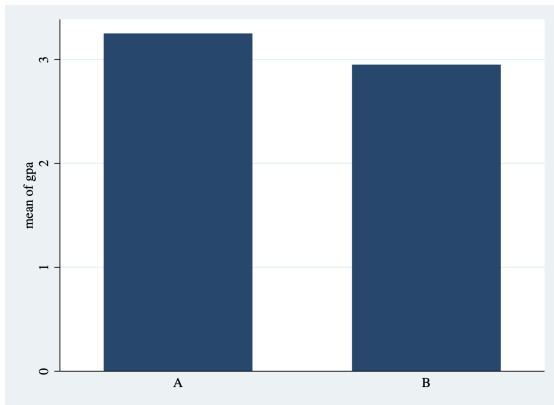
```
graph bar (stat) yvar, over(xvar)
```

- ▶ `(stat)` will be `(mean)` if you want the average value of the y-variable to be display. There are also other options in the help file
- ▶ `over(xvar)` will be the variable that defines the groups on the horizontal axis

Simple Bar Graph in Stata

- Now let's use the `graph bar` command to construct a graph that shows average gpa over different classrooms

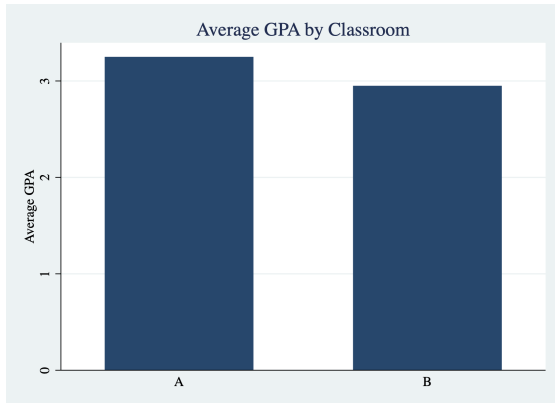
```
graph bar (mean) gpa, over(school)
```



Descriptive Titles

- As always need descriptive titles and properly labeled axes

```
graph bar (mean) gpa, over(school) ///  
  title("Average GPA by Classroom") ///  
  ytitle("Average GPA")
```



Bar Graphs

- ▶ There are many further options when constructing bar graphs
- ▶ Sometimes, the option you are looking for isn't available, but there are still ways to construct the required bar graph
- ▶ In other words, you may need to use some **data wrangling** to get your data in the right format to construct a bar graph

Collapse

- ▶ The `collapse` collapses your data to a different unit
- ▶ Imagine you have individual-level data on income, but you only care about average income by state for your analysis
- ▶ The `collapse` command will allow you to quickly transform your data to the state level
- ▶ This is very useful when trying to understand/report summary statistics

Basic Syntax

- ▶ The basic syntax for the collapse command is

```
collapse (stat) varlist1, by(varlist2)
```

- ▶ (stat) will be some statistic. The default is the mean
- ▶ varlist1 is a variable (or list of variables) that you would like to retrieve the mean for
- ▶ varlist2 is a variable (or list of variables) will define the groups over which the statistic is calculated

Basic Example

- Consider the dataset below (which is made up)

	worker_id	weekly_income	age	state
1	1	2510	22	CA
2	2	2278	44	CA
3	3	2504	51	CA
4	4	1272	49	CA
5	5	2806	57	CA
6	6	1188	62	AZ
7	7	2149	54	AZ
8	8	1745	41	AZ
9	9	1547	24	AZ
10	10	1780	24	AZ

- We want to transform this into a dataset that has the average weekly income and age for “CA” and “AZ”

Using collapse

- ▶ To retrieve our summary statistics, we type

```
collapse (mean) weekly_income age, by(state)
```

- ▶ In words, convert data so that there is a single observation per state, with average values of weekly income and age as variables

Resulting Dataset

- ▶ Our dataset has now been transformed into

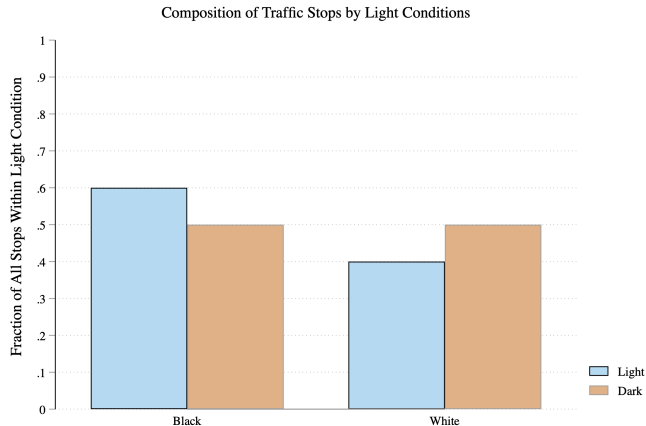
	state	weekly_income	age
1	AZ	1681.8	41
2	CA	2274	44.6

- ▶ Note: this fundamentally changes the level of observation. There is no quick “undo” to get back your individual-level dataset
- ▶ To retrieve the original dataset you would need to reload it into Stata

Our goal

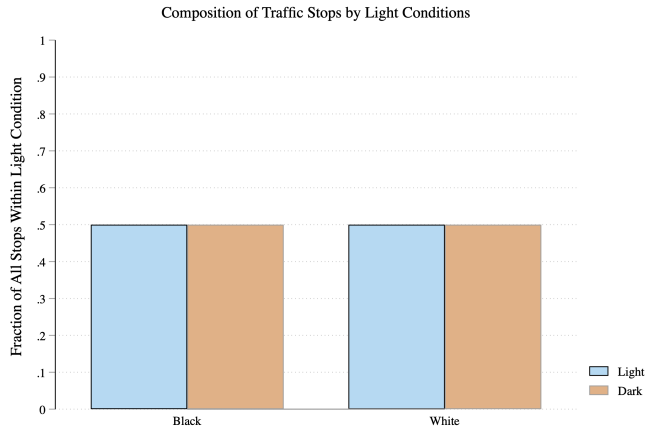
- ▶ We want to construct a data visualization that will allow us to see clearly the results of the veil of darkness test
- ▶ Let's look at some hypothetical examples before we begin so that we know what our end goal should look like

Example of Discriminatory Police Behavior



- Discriminatory because stops of Black drivers fall when it is dark, when officers are less able to discern race

Example with No Evidence of Discrimination



- No evidence of discrimination because stop fractions do not depend on whether it is light or dark

Comparison

- ▶ We need to compare the composition of traffic stops in times of sunlight vs. darkness
- ▶ We can use the `collapse` command to create a dataset of these statistics
- ▶ To start, let's create a dataset that has the number of stops by race, for both daylight and darkness

Collapse with Count

- ▶ To count the number of observations per race for both daylight and darkness, we can use the `count` option of the `collapse` command
- ▶ `count` will count the number of observations with non-missing values of a variable
- ▶ To begin, let's create a variable (that will eventually store our counts) that is non-missing for all observations

```
gen obs_count = 1
```

Collapse

- Now we can collapse the data

```
collapse (count) obs_count, by(subject_race dark)
```

	subject_race	dark	obs_count
1	Asian/Pacific Islander	0	375
2	Asian/Pacific Islander	1	372
3	Black	0	589
4	Black	1	495
5	Hispanic	0	1528
6	Hispanic	1	1319
7	Other	0	319
8	Other	1	242
9	White	0	1652
10	White	1	1437

Fractions Instead of Counts

- ▶ We really want the fraction of stops for each race by daylight and darkness
- ▶ Lucky for us, there is a command (`egen`) that will help us here
- ▶ `egen` stands for extensions to generate, and is very useful when trying to create more variables
- ▶ If you ever want to create a variable, but can't figure out exactly how, type `help egen` to see if it is possible with the `egen` command

Total Stops in Daylight vs. Darkness

- ▶ The following code will calculate the total stops in daylight vs. darkness

```
bysort dark: egen total_stops = total(obs_count)
```

- ▶ `bysort dark:` tells Stata that any code that comes after the `:` should be done separately for values of the `dark` variable
- ▶ `egen total_stops = total(obs_count)` will create a new variable that is equal to the sum total of the variable `obs_count`
 - ▶ In other words, the total number of stops in daylight and darkness

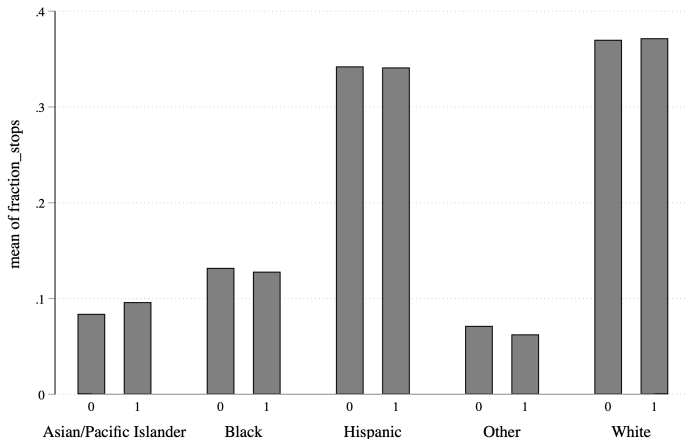
Fraction of Stops

- ▶ Now that we have (1) total stops by race in daylight vs. darkness and (2) total stops overall in daylight vs. darkness we can create our required variables

```
gen fraction_stops = obs_count/total_stops
```

- ▶ Now we can use this variable to show the composition of traffic stops (by race) both in the daylight and darkness

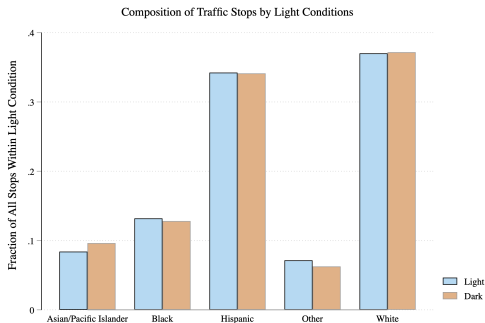
```
graph bar fraction_stops, over(dark) over(subject_race)
```



- This is a good start, but we need to make this graph more easily viewable/interpretable for the reader

Improved Graph (See Video 3.14 for details)

```
graph bar fraction, asyvars ///  
    over(dark, relabel(1 "Light" 2 "Dark")) over(subject_race) ///  
    title("Composition of Traffic Stops by Light Conditions") ///  
    ytitle("Fraction of All Stops") ///  
    bar(1, fcolor(sky)) ///  
    bar(2, fcolor(vermillion))
```



Summary of What We Have Learned

- ▶ The veil of darkness test is one tool used to detect racial discrimination in traffic stops
- ▶ Like any empirical design, the test has assumptions and limitations
 - ▶ It is not a fatal flaw that a given set of research has limitations. All research has limitations and it is important to think carefully about them

Conclusion

- ▶ We have barely scratched the surface of the questions that can be answered with the Stanford Open Policing Project (See Video 3.15 for more applications)
- ▶ Main takeaway from this module: data can be a powerful tool that informs policy
- ▶ The Stanford Open Policing Project is one effort to improve transparency in policing by making data publicly available to researchers and journalists